PyGameMaker — Tutoriel 5 : Sokoban — Guide de l'enseignant·e

Document d'accompagnement du tutoriel intégré (**Aide > Tutoriels > Sokoban : Puzzle Pousse-Caisses**, 4 pages) et de la fiche élève et de la feuille d'exercices correspondantes. Les élèves doivent avoir terminé le Tutoriel 2 ou 3 (événements, collisions, objets solides).

Vue d'ensemble

Les élèves construisent un puzzle de Sokoban en trois phases : un joueur qui se déplace sur une grille, des caisses qu'on peut pousser, et des cibles avec un retour visuel. Notions nouvelles : le **déplacement sur grille** (pas fixes de 32 pixels), l'événement **Touche pressée**, **Si peut pousser**, le **changement de sprite** avec un événement Pas, et la **conception de niveau**.

> La vraie leçon, c'est la conception de puzzles. Demandez aux élèves de résoudre les niveaux des autres : un niveau avec moins de cibles que de caisses, ou avec une caisse dans un coin, ne peut pas être résolu.

Durée suggérée (45 à 60 minutes)

Le tutoriel annonce 20 à 25 minutes pour un élève à l'aise ; en classe, prévoyez plus.

Séquence	Durée	Ce qui se passe
Rappel et hypothèses	5 min	Montrez un niveau de Sokoban ; demandez quelles sont les règles du jeu
Phase 1 : joueur et murs	10-15 min	Pages 1-2
Phase 2 : pousser des caisses	10-15 min	Page 3
Phase 3 : cibles et contrôleur	15 min	Page 4 ; inclut la question de l'ordre de placement ci-dessous
Échange et résolution	5 min	Les élèves jouent les niveaux des autres
Feuille d'exercices / ticket de sortie	5 min	Parties A à C de la feuille

Pour une séance de 45 minutes, arrêtez-vous après la phase 2 et terminez la phase 3 la fois suivante.

Déroulé étape par étape et difficultés fréquentes

Phase 1 : joueur et murs

- Utilisez des événements **Touche pressée** (une fois par appui), pas *Clavier* (maintenue). Avec des touches maintenues, le joueur courrait.
- *Déplacer grille* de taille **32** correspond à la grille de la salle, qui est de 32. La salle fait 320×320, et l'accrochage à la grille 32×32 dans l'éditeur de salle garde les objets alignés ; des objets non alignés donnent un mouvement bizarre.
- Le joueur a besoin de **En collision avec obj_wall** avec *Arrêter le mouvement*, et obj_wall doit être Solide.

Phase 2 : pousser des caisses

- La poussée se fait dans l'événement de collision **du joueur** : *Si peut pousser (facing)* puis *Pousser l'autre instance 32*. Si la poussée est bloquée (mur ou autre caisse derrière), le joueur s'arrête.
- La caisse doit être Solide et a aussi besoin de son propre événement **En collision avec obj_wall** avec *Arrêter le mouvement*.
- Une caisse contre un mur (ou dans un coin) au départ est coincée ; dites aux élèves de ne pas placer de caisses dans les coins.

Phase 3 : cibles et contrôleur

- obj_target ne doit **pas** être Solide, sinon les caisses et le joueur ne pourraient jamais s'y tenir.
- L'événement Pas de la caisse vérifie *Si en collision avec obj_target* et alterne entre spr_crate_ok et spr_crate.
- **L'ordre de dessin est l'ordre de placement.** La salle dessine les instances dans l'ordre où elles ont été placées, et l'éditeur de salle n'a ni réglage de profondeur ni « amener au premier plan ». Une cible placée après le joueur et les caisses est dessinée par-dessus : la caisse devient bien verte, mais elle est cachée sous la marque rouge, et le joueur semble disparaître sur une cible. Le tutoriel demande maintenant de placer les cibles d'abord, puis de supprimer et replacer le joueur et les caisses. Surveillez ce point : cela ressemble à « le retour visuel vert ne marche pas ».
- Le texte de consigne utilise la couleur de dessin par défaut, qui est noire ; il se trouve sur la rangée de murs du haut de la salle, donc il reste lisible. Utilisez *Définir la couleur de dessin* si vous le déplacez sur une zone noire.
- Le tutoriel n'a pas de message de victoire ; les élèves voient qu'ils ont réussi quand toutes les caisses sont vertes. Une vérification de victoire (« toutes les caisses sont sur des cibles ») est un bon défi.

> **Projets de référence.** Téléchargez les projets par étapes sur le wiki (05_sokoban_checkpoints.zip) : un projet pour la fin de chaque phase, avec le niveau d'exemple du tutoriel. Donnez à un élève bloqué la phase précédente.

Vocabulaire introduit

Terme	Ce que cela signifie ici
Déplacement sur grille	Se déplacer d'une distance fixe (32 pixels) par appui sur une touche
Touche pressée	Un événement qui se déclenche une fois quand une touche est enfoncée
Si peut pousser	Une vérification que la place derrière la caisse est libre
Changement de sprite	Changer l'image d'un objet avec *Définir le sprite*
Conception de niveau	Décider où vont les murs, caisses et cibles pour que le puzzle soit résoluble

Questions de discussion

- Pourquoi un événement par appui de touche, et non par image ?
- Qu'est-ce qui rend un niveau de Sokoban impossible ? (Une caisse dans un coin, trop peu de cibles.)
- Comment le jeu pourrait-il savoir que le puzzle est résolu ?

- Que change-t-on si les cibles sont Solides ? Essayez.

Différenciation

- **Soutien :** donnez le projet de la phase 2 ; utilisez le niveau d'exemple du tutoriel pour que tout le monde ait un puzzle résoluble.
- **Approfondissement :** compteur de déplacements ; plusieurs salles avec des niveaux plus difficiles ; une touche pour annuler ; un message « niveau terminé ».

Corrigé de la feuille d'exercices

Partie A : 1-C, 2-A, 3-D, 4-B.

Partie B : 1. obj_player. 2. obj_crate. 3. obj_crate (son événement Pas). 4. obj_controller. 5. obj_wall (et les caisses Solides bloquent le joueur).

Partie C :

1. Les caisses et le joueur ne pourraient pas entrer dans la case d'un objet Solide, donc rien ne pourrait jamais atteindre une cible.
2. Le joueur ne bouge pas et la caisse reste : *Si peut pousser* trouve la place derrière la caisse bloquée, et exécute donc *Arrêter le mouvement*.
3. Non, un niveau a besoin d'au moins autant de cibles que de caisses. Comptez toujours et résolvez votre propre niveau avant de le partager.

Parties D et E : réalisation et bilan.

Grille d'évaluation : Sokoban

Niveau	Ce que montre le jeu
4 - Complet	Déplacement sur grille ; les murs arrêtent joueur et caisses ; les caisses ne sont poussées que si la place derrière est libre ; les caisses deviennent vertes sur les cibles ; recommencer avec R ; un niveau résoluble avec autant de caisses que de cibles
3 - Fonctionnel	Déplacement, murs et poussée fonctionnent ; le retour visuel ou le redémarrage manque
2 - À moitié	Le joueur se déplace sur la grille, mais les caisses ne peuvent pas être poussées
1 - Commencé	Joueur et murs existent ; le déplacement n'est pas d'une case par appui

Liste de vérification de fin de séance

- ☐ Chaque élève a un projet enregistré (Ctrl+S)
- ☐ Chaque élève a atteint au moins la phase 2 (une caisse peut être poussée)
- ☐ Notez qui aura besoin du projet par étapes la prochaine fois